RIKO'S

Sistema de gestión integral para el Restaurante Riko's

Guía del equipo: cómo hemos trabajado, qué está hecho y qué falta

Estado al 11 de octubre de 2026. Esta guía resume el repositorio para quien lo recibe por primera vez. La fuente de verdad sigue siendo el repositorio: **ESTADO.md** (estado actual), **CLAUDE.md** (convenciones) y la carpeta **docs/**.

Dato	Valor
Qué es	Sistema multilocal para un restaurante criollo peruano con pensionistas (estudiantes con cuenta corriente). Cliente real.
Alcance total	11 épicas, 72 historias de usuario, 15 sprints (0 a 14)
Avance	Sprints 0 a 6 completos (falta solo la prueba con la impresora real). Sprint 7 en curso: 1 de 5 entregas. 23 historias construidas de 64.
Pruebas automáticas	Más de 600 (la última suite completa dio 600, sin contar las 16 de cajas)
Base de datos	40 migraciones (V1 a V40), PostgreSQL
Stack	Java con Spring Boot 4 y Spring Modulith, React 19 + TypeScript + Vite, PostgreSQL, Docker

1. Qué problema resuelve

Hoy el restaurante lleva todo en cuadernos. RIKO'S ataca cinco problemas:

- **La cuenta corriente del pensionista:** consumos, abonos, límite de crédito, cierre mensual con arrastre y un portal para el estudiante y su apoderado.
- **La venta en cuatro canales:** mesa, para llevar, delivery propio y recojo programado, con mesas mixtas donde conviven clientes normales y pensionistas.
- **El costo real del plato:** recetas con subrecetas, costeo por última compra, producción, Kardex y conteos físicos.
- **La continuidad sin Internet:** el local debe poder vender, comandar, cobrar y cargar consumos aunque se caiga la conexión, y sincronizar después sin duplicar.
- **El control administrativo:** auditoría de descuentos, anulaciones, ajustes y cierres, y reportes exportables.

Arquitectura en una línea

Un repositorio con tres servicios desplegables: **gateway** (puerta de entrada), **servicio-operacion** (monolito modular con toda la operación: identidad, catálogo, menú, pensionistas, venta, caja) y **servicio-facturacion** (comprobantes electrónicos, el único microservicio porque depende de un tercero). Más el frontend **rikos-web**. Dentro de operación, los módulos no se tocan entre sí salvo por interfaces explícitas que verifica una prueba automática (Spring Modulith).

2. Cómo hemos estado trabajando

Trabajamos con un método propio llamado **spec-driven** (especificar antes de programar), descrito en *docs/METODO-spec-driven.md*. La regla es: no se escribe código de una funcionalidad hasta que su especificación está escrita y aprobada por el dueño del producto.

Los 7 pasos del método (carpeta docs/producto/)

Paso	Carpeta	Qué responde
1. PRD	prd/	Qué problema resuelve, para quién, qué entra y qué no. Incluye el backlog v3.
2. Flujos	flujos/	Las 72 historias de usuario con criterios de aceptación, una épica por archivo, y las 94 reglas de negocio.
3. UI/UX	ui-ux/	Mapa de pantallas y reglas de diseño.
4. TRD	trd/	Arquitectura y stack.
5. Modelo de datos	modelo-de-datos/	Tablas y reglas, partido en 15 archivos por bloque.
6. Plan	plan/	Los 15 sprints, el estado real de cada uno y la especificación de cada sprint (sprint-NN-...md).
7. Construcción	rikos-backend/doc/historias/	Qué se construyó de verdad, cómo se verificó y qué bugs aparecieron. Un archivo por historia.

El ciclo de cada sprint

- **1. Brainstorming con el dueño:** una decisión por vez, con opciones y una recomendación. Lo que dijo el dueño queda marcado como tal; lo que se asumió, como supuesto.
- **2. Especificación escrita** en docs/producto/plan/sprint-NN-...md: objetivo, decisiones con su razón, qué queda fuera, entregas pequeñas y verificables, pruebas y riesgos. El dueño la aprueba.
- **3. Construcción por entregas**, cada una con sus pruebas escritas antes o junto con el código (TDD real).
- **4. Verificación:** pruebas automáticas más revisión en el navegador, a 1280 px y a 375 px (móvil).
- **5. Documentación al cerrar:** una historia en rikos-backend/doc/historias/, las decisiones técnicas en RIKOS-decisiones-tecnicas.md (DT-NNN) y la actualización de ESTADO.md.

Quién hizo qué

El trabajo se hizo con dos sesiones de Claude en paralelo, coordinadas por mensajes: una de **diseño del POS** (diseña en un lienzo, es dueña del shell, el mapa de mesas y los salones) y otra de **backend y pantalla del pedido** (la de este documento). Cada una avisa antes de tocar archivos de la otra. El dueño del producto es el usuario: aprueba cada especificación y cada diseño.

Reglas que se respetan siempre

- **Nunca se commitea ni se hace push sin que el usuario lo pida.**
- **Flyway manda:** una migración ya aplicada nunca se edita; una corrección va en una migración nueva. Nunca DROP TABLE.
- **Identificadores ULID** de 26 caracteres, nunca autoincrement ni UUID aleatorio. Los montos van en **céntimos** enteros.
- **Resultado<T, E>** para todo flujo que puede fallar de forma esperada: los errores de negocio no son excepciones.
- **La documentación es parte del trabajo:** si no está en doc/historias/, una sesión nueva no puede entender qué se hizo.
- **Frontend:** el chequeo de tipos es *npx tsc -b --force*, no *tsc --noEmit* (el tsconfig raíz es de tipo solution y --noEmit no verifica nada).

3. Estado por sprint

Cada sprint termina con algo que se puede enseñar. Verde = hecho, amarillo = en curso, gris = pendiente.

Sprint	Objetivo	Qué se puede demostrar	Estado
0	Que las herramientas dejen de estorbar	El navegador llama al backend por el gateway y muestra datos de la base	Hecho
1	Entrar al sistema	Login real, cambio rápido de mozo por PIN, local activo visible	Hecho
2	El primer módulo del negocio	Alta y edición de productos, unidades y proveedores	Hecho (salvo US-01: 56 px y teclado único)
3	Control y trazabilidad	Un usuario sin permiso no ve el botón; la bitácora registra quién cambió qué	Hecho
4	Costos y menú	Receta con subrecetas y su costo por porción; semana de menú programada	Hecho (214 pruebas)
5	La cuenta del pensionista	Planes tarifarios, motor de precios, ficha del pensionista y su cuenta corriente	Hecho (325 pruebas)
6	Vender en mesa	Mapa de mesas, pedido por persona, comanda imprimible, anulaciones, reorganizar cuentas	Casi hecho: falta HT-10, la impresora real
6b	Rediseño del POS	Shell, Inicio, Órdenes activas, Pedido rápido, Historial, fotos de producto	Hecho en lo esencial (diseño de otra sesión)
7	Cobrar y cerrar caja	Cobro dividido, cargo al pensionista, cierre de turno con conciliación	En curso: entrega 1 de 5 hecha
8	El circuito del pensionista completo	Kiosco, pre-ticket, mozo, caja y cargo, punta a punta	Pendiente
9	Existencias reales	Una venta descuenta la receta y el Kardex lo explica	Pendiente
10	Que el local no dependa de Internet	Se corta la conexión, el local sigue vendiendo y todo sincroniza sin duplicar	Pendiente
11	Comprobantes electrónicos	Boleta emitida, aceptada y enviada por correo	Pendiente (bloqueado por datos de terceros)
12	Canales no presenciales	Un delivery desde el registro hasta la liquidación del motorizado	Pendiente
13	Autoservicio y cierre mensual	El pensionista consulta su cuenta desde el celular y recibe su estado mensual	Pendiente
14	Visibilidad	Panel con filtros y reportes exportables	Pendiente

El sprint 7 en detalle (donde estamos)

Cobrar y cerrar caja. Especificado con el dueño el 5 de octubre de 2026 (documento *docs/producto/plan/sprint-07-cobrar-y-cerrar.md*). Decisiones del dueño: varias cajas por local (se siembra una), solo cobra un cajero con turno abierto, el cobro es por líneas y puede mezclar clientes normales y pensionistas, efectivo con vuelto, descuentos por línea con permiso y motivo, reapertura solo dentro del turno.

Entrega	Qué incluye	Estado
1. Cajas y turnos	Tablas caja y turno_caja, abrir turno con fondo inicial, un turno abierto por caja y por cajero garantizado por la base de datos	Hecho (solo API, sin pantalla)
2. Cobro	Marcar líneas, pago dividido con vuelto, cierre parcial, cargo a la cuenta del pensionista, snapshot de precio y costo; 'Terminar pedido' queda solo para pedidos sin nada que cobrar	Pendiente (siguiente)
3. Descuentos y anulación en caja	Descuento por línea y anulación con permiso, motivo y bitácora	Pendiente
4. Cierre de turno	Efectivo esperado, conteo, diferencia, alerta desde S/ 0,01 sin bloquear, cierre inalterable	Pendiente
5. Reapertura	Reabrir una venta solo mientras el turno de origen sigue abierto	Pendiente

4. Estado por épica

El proyecto tiene dos numeraciones. El repositorio y el código usan las **11 épicas originales** (EP-01 a EP-11) y las historias de tres dígitos (US-001 a US-062). El **backlog v3** (recibido el 4 de octubre) las reagrupa en **5 épicas** (E1 a E5) con historias de dos dígitos (US-01 a US-64). La tabla de traducción está en *docs/producto/prd/04-traduccion-de-ids-v3.md*. Ojo: US-01 (v3) y US-001 (repo) no son la misma historia.

Épicas del repositorio (EP-01 a EP-11)

Épica	Qué cubre	Historias	Avance	Estado
EP-01	Fundaciones: locales, usuarios, permisos, PIN, auditoría, horarios	US-001 a 007	7 de 7	Hecho
EP-02	Catálogos, unidades, proveedores, recetas y costeo, motivos	US-008 a 012	5 de 5	Hecho
EP-03	Menú normal y extra, programación semanal, agotados	US-013 a 015	3 de 3	Hecho
EP-04	Pensionistas: ficha, planes, cuenta corriente, kiosco, portal, abonos	US-016 a 026	3 de 11	En curso: faltan kiosco, abonos, portal, cierre mensual
EP-05	POS, mesas y cocina	US-027 a 031	5 de 5	Hecho (falta HT-10, impresora real)
EP-06	Caja, cobro y facturación electrónica	US-032 a 037, 062	0 de 7 (US-032 entrega 1 hecha)	En curso
EP-07	Delivery y recojo programado	US-038 a 042	0 de 5	Pendiente
EP-08	Inventario, producción y compras	US-043 a 051	0 de 9	Pendiente
EP-09	Panel de administración y reportes	US-052 a 057	0 de 6	Pendiente
EP-10	Offline, sincronización, respaldo y seguridad	US-058 a 061	0 de 4	Pendiente
EP-11	Habilitantes técnicas	HT-01 a HT-10	6 de 10	En curso: faltan HT-04 outbox, HT-08, HT-09, HT-10

Épicas del backlog v3 (E1 a E5, 64 historias)

Épica v3	Causa del problema que ataca	Historias	Hechas	Estado
E1 Registrar los pedidos de venta	Pedidos en papel entre salón y cocina	14 (US-01 a 14)	8, más US-01 parcial	En curso: faltan recojo y delivery
E2 Controlar la cobranza	Consumos en cuadernos sin validar identidad, saldo ni local	18 (US-15 a 32)	3, más US-18 (turno) en curso	En curso: sprints 7, 8, 11 y 13
E3 Controlar el consumo de insumos	Compras y salidas sin relación con recetas ni ventas	14 (US-33 a 46)	5	En curso: catálogo y recetas hechos; falta inventario
E4 Continuidad del servicio	Inestabilidad de Internet	5 (US-47 a 51)	0	Pendiente
E5 Supervisar la operación	Acceso por roles y bitácora; reportes	13 (US-52 a 64)	7	En curso: fundaciones hechas; faltan los reportes

Total: **23 de 64 historias construidas**. Las que faltan son, sobre todo, caja y cobro (sprint 7), el circuito completo del pensionista, el inventario, el modo sin Internet, los comprobantes, el delivery y los reportes.

5. Qué hay construido, en concreto

Área	Qué funciona hoy
Acceso	Login con usuario y contraseña, PIN de mozo con bloqueo por intentos, local activo, roles y permisos granulares, bitácora de auditoría.

Área	Qué funciona hoy
Administración	Panel web para locales y sus horarios, usuarios, roles, productos, unidades, proveedores, recetas y motivos.
Menú	Menú normal y extra, programación semanal con estados por fecha, productos agotados por local.
Pensionistas	Planes tarifarios con versiones, parámetros de cobro, motor de precios, ficha con responsable y cuenta corriente de solo agregado.
POS	Mapa de mesas por salón, abrir pedido, asignar productos por persona, menú del día, extra libre, comanda con foto y reimpresión, anulaciones con motivo, mover, unir y separar mesas, trasladar productos, lista de pedidos, 'Terminar pedido', fotos de producto.
Caja	Cajas por local y apertura de turno con fondo inicial (solo API; falta la pantalla).

6. Qué falta y qué está abierto

Lo que sigue, en orden

- **Sprint 7:** entregas 2 a 5 (cobro, descuentos y anulación, cierre de turno, reapertura) y la pantalla de caja, que necesita diseño.
- **Sprint 8:** el circuito del pensionista completo (pre-ticket del kiosco, vinculación al pedido, abonos en efectivo).
- **Sprint 9:** existencias reales: inventario, compras, Kardex y descuento de receta al vender.
- **Sprint 10:** operación sin Internet, sincronización sin duplicados y respaldo.
- **Sprint 11:** comprobantes electrónicos (bloqueado hasta definir proveedor y datos fiscales).
- **Sprints 12, 13 y 14:** delivery y recojo, portal del pensionista y cierre mensual, panel y reportes.

Pendientes que no son código

- **HT-10:** prueba de impresión con la comandera real. Hoy la comanda es una vista imprimible desde el navegador; falta el hardware.
- **Precios reales:** los precios del plan, del táper y del menú, y el pensionista 'Luis Alberto Quispe Rojas', son datos de desarrollo inventados. Hay que reemplazarlos por los reales del dueño antes de cobrarle a alguien.
- **Para el dueño (sprint 7):** fondo inicial habitual, nombres de las cajas, tope de descuento por rol y motivos iniciales de descuento y anulación.
- **Tipo de entrega** de la comanda (US-06 CA6): falta saber qué valores quiere el dueño.
- **Código real del local:** 'sanisidro' es dato de desarrollo y es contrato del POS; definirlo antes del primer despliegue.

Deuda técnica conocida

- **Ciclo de módulos comun a identidad:** es el único test que falla (ArquitecturaDeModulosTest). Está reportado y no resuelto.
- **US-01 (POS con una mano):** el rediseño usa objetivos táctiles de 44 px y los criterios del backlog v3 piden 56 px; las cantidades usan botones en vez del teclado numérico compartido.
- **'Terminar pedido'** cierra sin cobro y sin motivo; es provisional y el sprint 7 lo acota. Esos pedidos no cuentan como ventas.
- **Inmutabilidad de la bitácora** solo garantizada por la aplicación, no por la base de datos.
- **Limpieza de bloqueo_pin:** no hay proceso programado.

Comparación con el backlog v3

El v3 es el v2.0 reordenado, no reescrito: los criterios de aceptación, prioridades y las 94 reglas de negocio son los mismos. El único cambio real es que la tarea técnica HT-07 pasó a ser la historia US-01. La comparación completa está en *docs/producto/plan/04-comparacion-backlog-v3.md* y las decisiones distintas del v3 en *docs/producto/prd/05-enmiendas-al-backlog-v3.md*.

7. Cómo empezar a trabajar

Levantar el entorno

- Leer **docs/EMPEZAR.md**: tiene el paso a paso, los usuarios y PIN de desarrollo y una tabla de errores comunes.
- Base de datos y servicios auxiliares: *docker compose up -d* desde rikos-backend/ y esperar a que PostgreSQL esté 'healthy'. PostgreSQL de este proyecto usa el puerto **5433**.
- Backend: *../mvnw spring-boot:run* **desde la carpeta del módulo** (servicio-operacion y gateway), nunca desde la raíz con -pl.
- Frontend: *npm run dev* en rikos-web (puerto 5173). El panel de administración está en /admin y el POS en /pos.
- Si algo falla con 'Connection refused', primero *docker ps*: Docker se cae seguido en Windows con WSL2.

Orden recomendado de lectura

#	Documento	Para qué
1	README.md	El pitch y la arquitectura en una página.
2	ESTADO.md	En qué punto está el proyecto hoy (se actualiza al cerrar cada sesión).
3	CLAUDE.md	Las convenciones que más se rompen.
4	docs/METODO-spec-driven.md	El método de trabajo.
5	docs/producto/plan/README.md	El estado real por sprint.
6	docs/producto/plan/sprint-07-cobrar-y-cerrar.md	Un ejemplo completo de especificación de sprint.
7	rikos-backend/doc/RIKOS-decisiones-tecnicas.md	Por qué se decidió cada cosa técnica (DT-001 en adelante).
8	rikos-backend/doc/historias/	Qué se construyó y cómo se verificó, una historia por archivo.

Cómo agregar algo nuevo

- Una funcionalidad nueva se **especifica primero** (docs/producto/plan/) y se aprueba; después se programa.
- El caso de uso lleva su **prueba antes o junto con el código**; cada endpoint lleva su prueba de acceso por rol.
- Al cerrar, se escribe la historia en rikos-backend/doc/historias/ y se actualizan ESTADO.md y el README del plan.
- Una columna o tabla nueva va en una **migración nueva** (V41 en adelante), nunca editando una anterior.